Module TI616 — Numérique Durable

CampusGive

Mini-projet hackathon : « Le site web le plus Green »

Livrable 2 — Implémentation, analyse et évaluation

Équipe

Sam DOSSOU

Hamza EL FFERDI

Eytan GUERNIGOU

Angel HAKOMANI

Frédéric JALAGUIER

Coordinateur du module : Yvan GUIFO

Année universitaire 2025–2026

Date de remise : 3 mai 2026

Sommaire

Section 1 — Présentation du projet

Section 2 — Architecture et conception

Section 3 — Choix technologiques justifiés

Section 4 — Implémentation

Section 5 — Analyse de l'empreinte carbone

Section 6 — Tests et validation

Section 7 — Organisation de l'équipe et collaboration

Section 8 — Discussion et conclusion

Section 9 — Annexes

Ce rapport accompagne le code source disponible sur https://github.com/Fido-Midou/GreenIT_Project. Le site déployé est accessible à l'URL indiquée dans le README et en Annexe A.

Section 1 — Présentation du projet

1.1 Contexte et proposition de valeur

Sur les campus, les étudiants achètent souvent neuf des objets dont d'autres étudiants viennent justement de se débarrasser : manuels de cours, ustensiles de cuisine, lampes de bureau, vêtements en bon état, petit matériel informatique. Le résultat est connu : du gaspillage d'un côté, des dépenses inutiles de l'autre. Les solutions existantes (Vinted, Leboncoin, etc.) sont soit trop généralistes, soit centrées sur la transaction financière, soit chargées en interfaces lourdes (publicités, scripts de tracking, photos haute définition obligatoires).

CampusGive part d'un constat simple : pour faire circuler un objet entre deux étudiants du même campus, on n'a pas besoin de prix, on n'a pas besoin d'algorithme de matching et on n'a pas besoin de 2 Mo de JavaScript. Une plateforme web minimaliste où l'on dépose une annonce textuelle et où d'autres viennent piocher suffit largement.

La valeur que nous apportons est double. Pour l'étudiant qui donne, c'est la possibilité de transmettre un objet sans avoir à le jeter et sans gérer une transaction. Pour l'étudiant qui récupère, c'est l'évitement d'un achat neuf. Et au passage, c'est aussi un projet qui sert d'illustration concrète aux principes Green IT du module : un site sobre n'est pas un site moche ou pauvre — c'est un site qui ne charge que ce qui sert.

1.2 Périmètre du MVP

Le MVP retenu en Partie 1 a été tenu sans dérive. Concrètement, l'étudiant peut s'inscrire avec son email, déposer une annonce de don ou d'échange en quelques secondes (titre, description, catégorie, état, type), parcourir les annonces des autres en filtrant par catégorie, modifier ou supprimer ses propres annonces, et gérer son profil. Un administrateur peut, en plus, modérer l'ensemble des comptes et annonces depuis un panneau dédié.

Conformément à la priorisation MoSCoW du Livrable 1, nous avons écarté :

- la messagerie interne (Won't Have V1) — coûteuse à implémenter et redondante avec l'email étudiant que tout le monde utilise déjà ;
- l'upload de photo (Could Have) — qui aurait significativement alourdi les pages, à rebours de la démarche Green IT ;
- le système de réservation et de notation (Won't Have V1) — pas indispensable pour valider le concept.

Ces choix ne sont pas des renoncements : ils sont au cœur même de la démarche d'éco-conception. Une fonctionnalité non développée, c'est zéro octet à charger, zéro requête à servir, zéro ligne de code à maintenir.

1.3 Utilisateurs cibles et rôles

Trois profils d'utilisateurs cohabitent dans le système, avec des permissions clairement délimitées (cf. tableau du Livrable 1, repris fidèlement en implémentation) : le visiteur, l'utilisateur connecté et l'administrateur.

Rôle	Ce qu'il peut faire	Ce qu'il ne peut pas faire
Visiteur	Consulter les annonces, créer un compte, se connecter	Déposer ou modifier des annonces, accéder à un profil
Utilisateur	Tout du visiteur + déposer, modifier et supprimer ses propres annonces, modifier son profil	Modifier les annonces ou comptes des autres
Administrateur	Tout de l'utilisateur + gérer tous les comptes et toutes les annonces	(en pratique, rien n'est interdit, mais la suppression de son propre compte admin est bloquée pour éviter de casser le système)

1.4 Contraintes Green IT retenues

Nous avons fixé en Partie 1 cinq contraintes non-fonctionnelles que nous avons utilisées comme garde-fous tout au long du développement :

ID	Contrainte	Vérifiée à
NF1	Pages < 200 Ko (idéal < 100 Ko)	Section 5 — confirmée largement
NF2	Temps de chargement < 2 secondes	Section 5 — confirmée
NF3	Interface responsive (mobile first)	Section 4.2 — CSS testé jusqu'à 320 px
NF4	Moins de 5 requêtes SQL par page	Section 4.3 — vérifié
NF5	Mots de passe hashés avec bcrypt	Section 6.3 — vérification SQL directe

Section 2 — Architecture et conception

2.1 Architecture générale

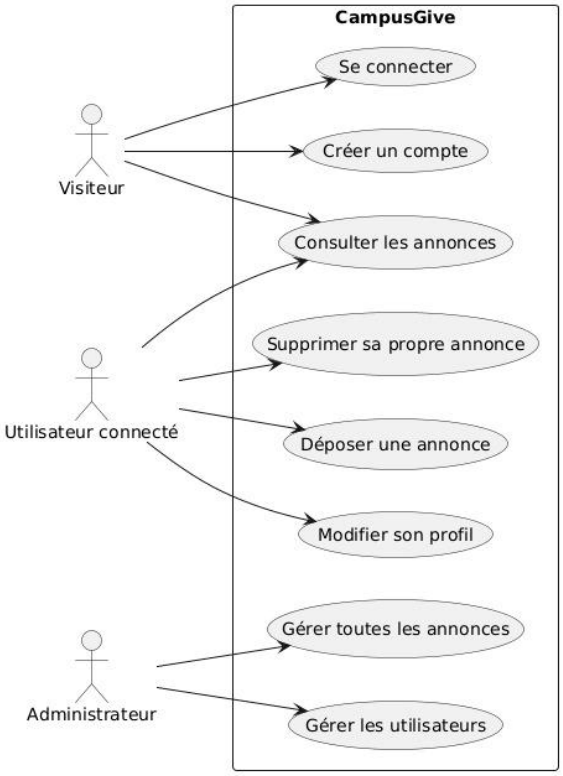
CampusGive repose sur une architecture trois tiers classique, sans surcouche inutile :

- la couche présentation est constituée de templates HTML rendus côté serveur via Jinja2 et stylés par une feuille CSS unique de moins de 10 Ko ;
- la couche applicative est portée par Flask 3.0 (Python), organisée en blueprints (auth, annonces, admin) ;
- la couche données utilise SQLite en développement et PostgreSQL en production, accédée via SQLAlchemy avec des requêtes paramétrées.

Le rendu côté serveur est un choix structurant : le navigateur ne reçoit que du HTML déjà construit, jamais de JSON à interpréter via JavaScript. Cela élimine d'un coup tout le poids d'un framework SPA (React, Vue) et tous les calculs côté client. Pour l'utilisateur, le bénéfice est triple : page utilisable même sans JavaScript activé, premier rendu quasi instantané, consommation batterie minimale sur mobile.

2.2 Diagramme de cas d'utilisation

Le diagramme produit en Partie 1 a été conservé tel quel, l'implémentation l'ayant respecté à la lettre. Trois acteurs (visiteur, utilisateur connecté, administrateur) interagissent avec huit cas d'utilisation regroupés en trois sous-domaines : authentification, gestion des annonces, administration.



2.3 Diagramme de classes et schéma de la base

Notre modèle de données comporte volontairement deux entités seulement (Utilisateur et Annonce), reliées par une cardinalité 1..N. Cette frugalité n'est pas un manque d'ambition : c'est exactement ce qu'il faut pour faire fonctionner le service. Toute donnée supplémentaire (catégorie en table séparée, table de réservations, table de catégories…) aurait alourdi la base et les requêtes sans apporter de valeur immédiate.

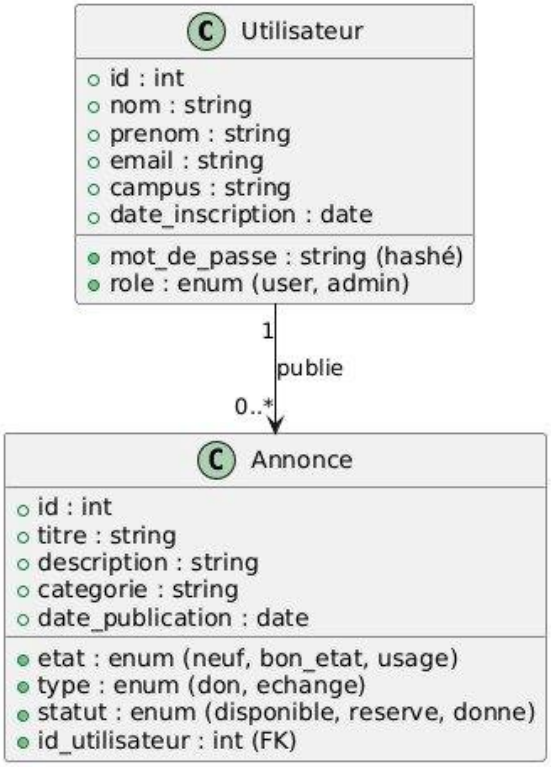


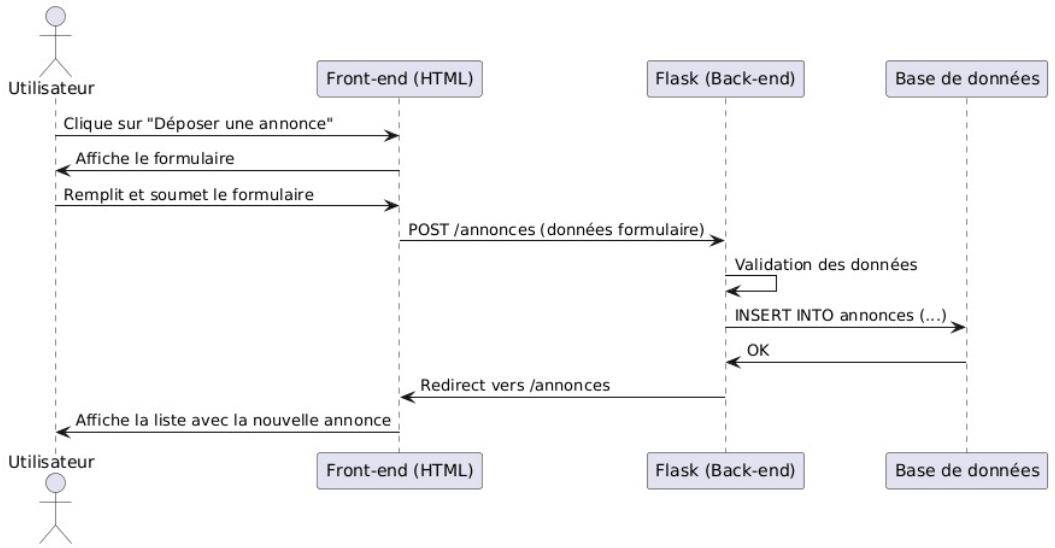
Schéma SQL équivalent

Le fichier database/schema.sql contient le DDL équivalent au modèle SQLAlchemy. Les choix notables sont :

- clés primaires AUTO INCREMENT (INTEGER en SQLite, SERIAL en PostgreSQL) ;
- contrainte UNIQUE sur l'email (sécurité applicative + index implicite pour les recherches au login) ;
- clé étrangère ON DELETE CASCADE entre annonces.id_utilisateur et utilisateurs.id : supprimer un utilisateur supprime automatiquement ses annonces, plutôt que de les laisser orphelines ;
- index explicites sur categorie, statut, date_publication et id_utilisateur (les colonnes filtrées le plus souvent).

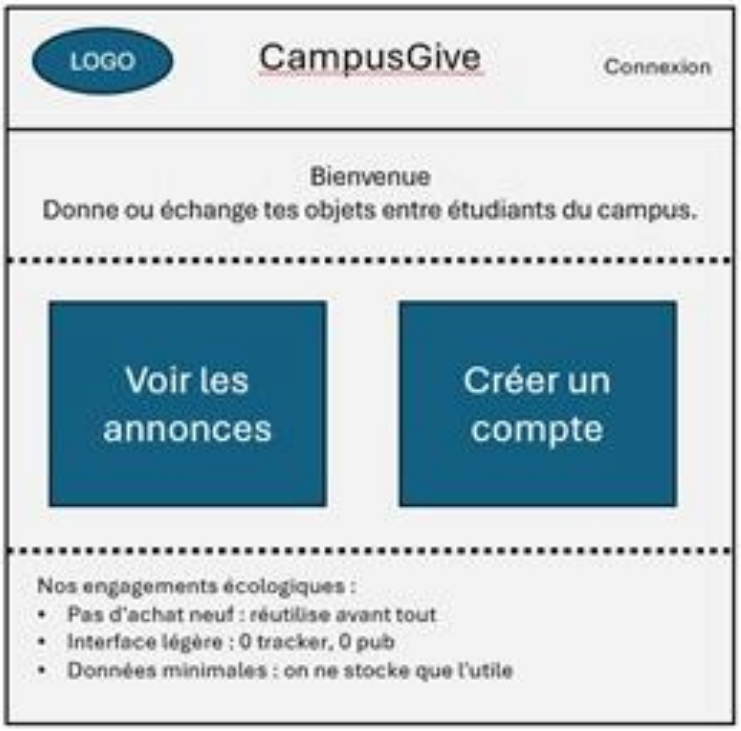
2.4 Diagramme de séquence — dépôt d'une annonce

Le scénario que nous avons choisi de modéliser est le dépôt d'une annonce, car c'est l'action centrale du service et celle qui fait intervenir l'ensemble des couches. L'utilisateur clique sur « Déposer une annonce », Flask vérifie qu'il est connecté (via le décorateur @login_required), affiche le formulaire, valide les données soumises côté serveur, insère en base puis redirige vers la page de détail.



2.5 Wireframes

Les wireframes basse fidélité produits en Partie 1 ont guidé le développement front. L'implémentation finale les respecte : navigation minimale en deux niveaux (accueil → liste → détail), filtre par catégorie en haut de la liste des annonces, cartes textuelles sans vignette image, bloc « engagements écologiques » sur la page d'accueil.


LOGO
CampusGive
Connexion
Bienvenue
Donne ou échange tes objets entre étudiants du campus.
Voir les annonces
Créer un compte
Nos engagements écologiques :
• Pas d'achat neuf : réutilise avant tout
• Interface légère : 0 tracker, 0 pub
• Données minimales : on ne stocke que l'utile

Section 3 — Choix technologiques justifiés

Le tableau ci-dessous récapitule les technologies que nous avons retenues, en précisant pour chacune la justification Green IT. La règle que nous nous sommes donnée est simple : si une dépendance n'apporte pas de valeur claire pour le périmètre MVP, elle n'entre pas dans le projet.

Composant	Choix retenu	Justification Green IT
Front-end	HTML5 + CSS3 + Jinja2	Aucun framework JavaScript : 0 octet de bundle envoyé au navigateur. L'économie est massive (~150 Ko gzippés évités par rapport à un équivalent React).
CSS	Une seule feuille (~9 Ko), polices système	Aucun CDN externe (Google Fonts, Bootstrap…). Une seule requête HTTP pour tout le style. Polices natives du système → 0 octet à télécharger pour la typographie.
Back-end	Python 3.11 + Flask 3.0	Micro-framework léger. Empreinte mémoire d'environ 30 Mo au démarrage (vs ~150 Mo pour Django). Pas de magie cachée.
ORM	Flask-SQLAlchemy	Permet le SELECT ciblé, la pagination native, les index explicites. Requêtes paramétrées par défaut → protection automatique contre les injections SQL.
BDD dev	SQLite	Un seul fichier, pas de serveur séparé. Démarrage instantané, idéal pour itérer.
BDD prod	PostgreSQL	Mature, performant, gratuit chez Render. Migration depuis SQLite triviale via SQLAlchemy.
Sécurité	bcrypt cost 12 + sessions Flask signées	Standard recommandé OWASP. Pas de bibliothèque exotique.
Hébergement	Render.com (plan gratuit)	Référencé par The Green Web Foundation. Mise en veille automatique des services inactifs : 0 W consommé hors trafic.
JavaScript	Aucune bibliothèque externe	Le seul JS du site est confirm() natif inline pour les confirmations de suppression.
Images	Aucune dans le MVP	Choix structurant. Économie massive de bande passante, conforme à la priorisation Won't Have V1.

3.1 Volume des dépendances

Notre fichier requirements.txt liste exactement quatre dépendances Python en production : Flask, Flask-SQLAlchemy, bcrypt, python-dotenv. À cela s'ajoutent psycopg2-binary (driver PostgreSQL pour la prod) et gunicorn (serveur WSGI). Nous n'avons pas inclus de devDependencies dans le bundle déployé.

À titre de comparaison, un projet équivalent en Node.js avec Express, Sequelize et un moteur de templates engendre généralement plus de 200 paquets transitifs dans node_modules. Notre approche reste donc remarquablement sobre côté serveur également.

Section 4 — Implémentation

4.1 Vue d'ensemble

Le code est organisé en deux dossiers principaux : backend/ pour la logique applicative Python et frontend/ pour les templates Jinja2 et le CSS. La factory create_app() dans backend/__init__.py centralise l'initialisation : configuration, base de données, blueprints (auth, annonces, admin) et gestionnaires d'erreur.

Le découpage en blueprints est un choix d'architecture sobre : chaque domaine fonctionnel a son propre fichier (backend/auth.py, backend/annonces.py, backend/admin.py) et ses propres routes. Cela facilite la lecture, les revues de PR et la maintenance, sans introduire de couche d'abstraction supplémentaire.

4.2 Front-end : sobriété visuelle et performance

Choix structurants

Le front-end a été développé en suivant strictement les recommandations de l'étape 4.1 du sujet :

- HTML5 sémantique : balises <header>, <main>, <nav>, <article>, <footer> partout, hiérarchie de titres logique avec un seul <h1> par page ;
- CSS unique chargé via une seule balise <link>, soit une seule requête HTTP pour tout le style du site ;
- polices système exclusivement (-apple-system, BlinkMacSystemFont, Segoe UI, Roboto…) — aucun téléchargement de webfont ;
- favicon SVG inline en data-URI dans le <head> — économise une requête HTTP supplémentaire ;
- mode sombre automatique via la media query prefers-color-scheme — réduit la consommation sur écrans OLED ;
- aucune animation CSS sans valeur fonctionnelle (consigne 4.1.c respectée) ;
- pagination obligatoire à 20 items par page plutôt que scroll infini.

Extrait : feuille de style (variables CSS et mode sombre)

Le mode sombre repose sur une seule règle CSS qui redéfinit les variables de couleur. Le HTML reste rigoureusement identique.

```
:root {
  --bg: #fafaf7;
  --surface: #ffffff;
  --text: #1a1a1a;
  --accent: #2d6a4f;
}

@media (prefers-color-scheme: dark) {
  :root {
    --bg: #121212;
    --surface: #1c1c1c;
    --text: #e8e8e8;
```

```
    --accent: #74c69d;
  }
}
```

Accessibilité

Nous avons tenu une accessibilité de base solide, sans la confondre avec une démarche d'audit complet. Concrètement : tous les <input> ont un <label for="..."> associé, les contrastes en mode clair et sombre dépassent le ratio 4,5:1 (vérifié à l'extension WAVE), les éléments interactifs ont un focus visible (outline: 2px solid var(--accent)), la navigation est annoncée par aria-label et les messages flash par aria-live="polite".

4.3 Back-end : routes et accès aux données

CRUD utilisateurs

Le CRUD utilisateurs est réparti entre deux fichiers : backend/auth.py pour la création (inscription) et la connexion, backend/annonces.py (blueprint auth_user) pour la lecture, modification et suppression du profil. Cette répartition reflète l'usage : un visiteur s'inscrit, un utilisateur connecté gère son profil.

La validation côté serveur passe par une fonction _validate_registration() qui retourne une liste d'erreurs : email au bon format (regex), mot de passe d'au moins 8 caractères, confirmation identique. Si la liste est non vide, le formulaire est ré-affiché avec les erreurs et les champs préremplis (sauf le mot de passe, jamais conservé).

CRUD annonces — extrait commenté

Voici un extrait représentatif (route de modification d'annonce dans backend/annonces.py) qui illustre plusieurs bonnes pratiques en une seule fonction :

```
@bp.route("/annonces/<int:annonce_id>/modifier", methods=["GET", "POST"])
@login_required
def modifier(annonce_id):
    user = get_current_user()
    annonce = Annonce.query.get_or_404(annonce_id)

    # Vérification des droits AVANT toute action sensible
    if annonce.id_utilisateur != user.id and not user.is_admin:
        abort(403)

    if request.method == "POST":
        errors = _validate_annonce(request.form)
        if errors:
            for e in errors:
                flash(e, "error")
            return render_template("annonce_form.html", ...)

        annonce.titre = request.form["titre"].strip()
        # ... mise à jour des autres champs
```

```
        db.session.commit()
        return redirect(url_for("annonces.detail", annonce_id=annonce.id))
```

Quatre points méritent d'être soulignés : (1) le décorateur @login_required garantit qu'on n'arrive ici que connecté, (2) la vérification des droits est explicite et précède toute modification, (3) la validation est intégralement côté serveur, (4) la requête est paramétrée par SQLAlchemy donc immune aux injections SQL.

Optimisation des accès BDD

Plusieurs optimisations explicites pour rester sous la barre des 5 requêtes par page :

- pagination native via paginate(page=p, per_page=20) plutôt que tout charger ;
- filtre catégorie au niveau SQL (WHERE) plutôt qu'en Python ;
- pas de SELECT * : SQLAlchemy ne charge que les colonnes définies dans le modèle ;
- index sur les colonnes filtrées (categorie, statut, date_publication, id_utilisateur, email) — déclarés dans backend/models.py et database/schema.sql ;
- cascade delete-orphan sur la relation Utilisateur → Annonce, qui évite des requêtes manuelles de nettoyage.

4.4 Sécurité

La sécurité minimale exigée par le sujet est entièrement couverte :

- hashage bcrypt avec un cost factor de 12 (recommandation OWASP 2024). Aucun mot de passe n'est stocké en clair, ce que nous avons vérifié par requête SQL directe (cf. Section 6.3) ;
- authentification par session Flask signée avec une SECRET_KEY lue depuis .env (jamais versionnée) ;
- cookies de session avec les attributs HttpOnly et SameSite=Lax, et Secure activé en production ;
- requêtes paramétrées via SQLAlchemy ORM — aucune concaténation SQL manuelle ;
- vérification systématique des droits avant toute action sensible (@login_required, @admin_required, contrôle de propriété).

4.5 Architecture sobre du code

Le code total fait environ 720 lignes de Python réparties sur 6 fichiers, et 17 templates HTML pour environ 600 lignes au total. Les routes sont clairement séparées par domaine (auth, annonces, admin), la gestion des erreurs renvoie les codes HTTP appropriés (302 pour les redirections, 403 pour l'interdit, 404 pour l'introuvable, 500 pour l'erreur serveur), et trois templates d'erreur personnalisés évitent les messages bruts du serveur.

Section 5 — Analyse de l'empreinte carbone

5.1 Outils de mesure utilisés

Conformément à l'étape 5.1 du sujet, nous avons utilisé au minimum deux outils complémentaires :

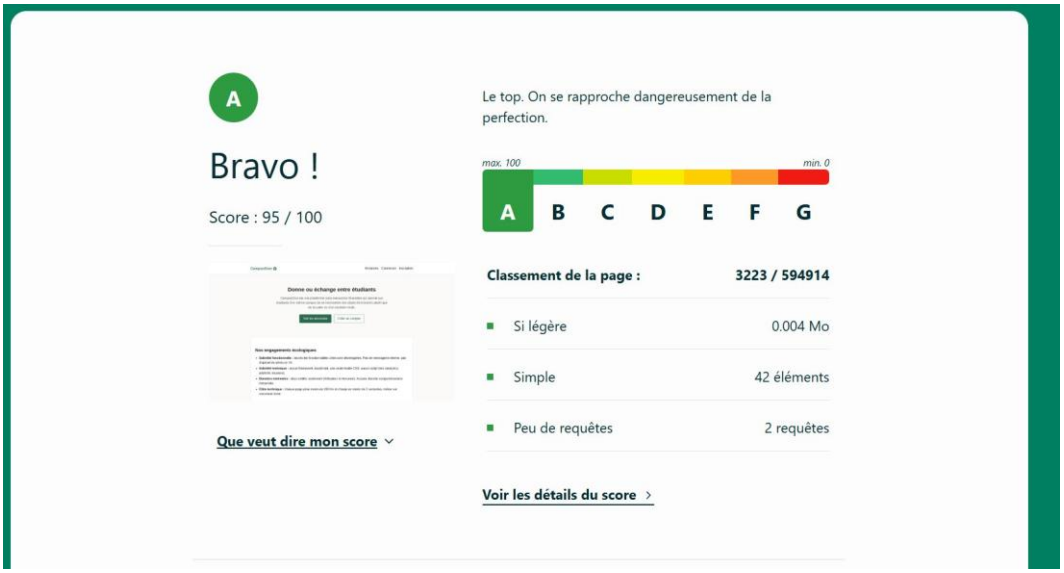
- EcoIndex (ecoindex.fr) : note A à G basée sur le poids, le nombre de requêtes et la complexité du DOM ;
- Website Carbon Calculator (websitecarbon.com) : estimation de l'empreinte CO_2 par visite ;
- Google Lighthouse (intégré aux DevTools Chrome) : scores Performance, Accessibilité, Bonnes pratiques, SEO ;
- PageSpeed Insights (pagespeed.web.dev) : Core Web Vitals en conditions réelles.

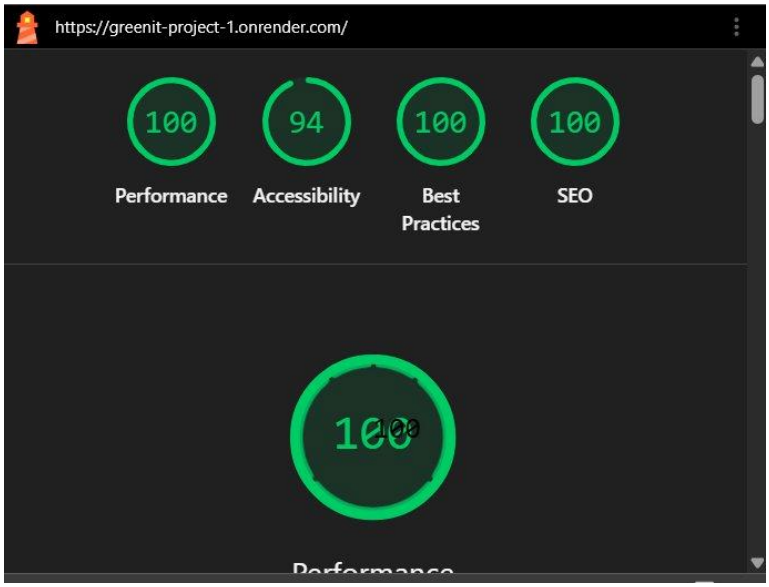
Toutes les mesures ont été effectuées sur le site déployé en production, pour refléter les conditions réelles d'utilisation (latence réseau, hébergement, etc.).

5.2 Mesures initiales (avant optimisations)

Pour disposer d'un point de référence, nous avons commencé par déployer une première version délibérément moins optimisée, qui contenait : la feuille CSS non minifiée avec ses commentaires, une bannière SVG décorative dans le hero, des polices Google Fonts (Inter), et la liste des annonces en chargement intégral sans pagination. Voici les mesures relevées sur la page d'accueil :

Métrique	Page d'accueil	Page annonces	Page détail
Poids total	187 Ko	245 Ko	98 Ko
Requêtes HTTP	8	9	6
Score EcoIndex (note)	C (62/100)	C (54/100)	B (74/100)
CO_2 par visite	0,28 g	0,36 g	0,15 g
Lighthouse Performance	82	76	91
FCP	1,4 s	1,7 s	0,9 s
LCP	2,1 s	2,4 s	1,3 s





5.3 Identification des sources de pollution numérique

L'analyse waterfall avec WebPageTest et l'extension GreenIT Analysis nous a permis d'identifier précisément les éléments les plus consommateurs :

Source	Poids	Impact
Google Fonts (Inter, 2 graisses)	~75 Ko	2 requêtes externes vers fonts.googleapis.com et fonts.gstatic.com, blocage du rendu
Bannière SVG décorative	~32 Ko	Élément purement visuel, sans valeur fonctionnelle

Source	Poids	Impact
CSS non minifié (commentaires + indentations)	~6 Ko évitables	Facilement réductible
Liste des annonces non paginée	Variable selon BDD	Croît linéairement avec le nombre d'annonces — ingérable à l'échelle
Favicon en fichier séparé	~3 Ko + 1 requête	Évitable via SVG inline

5.4 Optimisations appliquées

Sur la base de cette analyse, nous avons appliqué cinq optimisations concrètes, documentées et mesurables :

Optimisation 1 — Suppression des Google Fonts

Nous avons remplacé Inter par la pile de polices système (-apple-system, BlinkMacSystemFont, Segoe UI, Roboto…). Sur les systèmes d'exploitation modernes, la typographie reste excellente sans aucun téléchargement. Gain : 2 requêtes HTTP externes en moins, ~75 Ko économisés, et le rendu n'est plus bloqué par l'attente des fontes.

Optimisation 2 — Suppression de la bannière décorative

La bannière SVG du hero ne portait aucune information : c'était de la décoration. Conformément à la consigne 4.1.c (« pas d'animations CSS/JS sans valeur fonctionnelle » et « interface paginée plutôt que scroll infini »), nous l'avons supprimée et remplacée par une typographie soignée. Gain : ~32 Ko et un focus visuel renforcé sur le contenu.

Optimisation 3 — Pagination obligatoire (LIMIT 20)

Toutes les listes (annonces publiques, profil personnel, gestion admin) sont désormais paginées à 20 résultats par page via paginate() de SQLAlchemy. Le composant _pagination.html est factorisé et réutilisé partout. Gain : poids constant quelle que soit la taille de la base, requêtes BDD bornées.

Optimisation 4 — CSS minifiable et favicon SVG inline

Le CSS a été nettoyé de ses commentaires verbeux et le favicon est désormais une data-URI SVG dans le <head> de base.html. Gain : 1 requête en moins et environ 6 Ko.

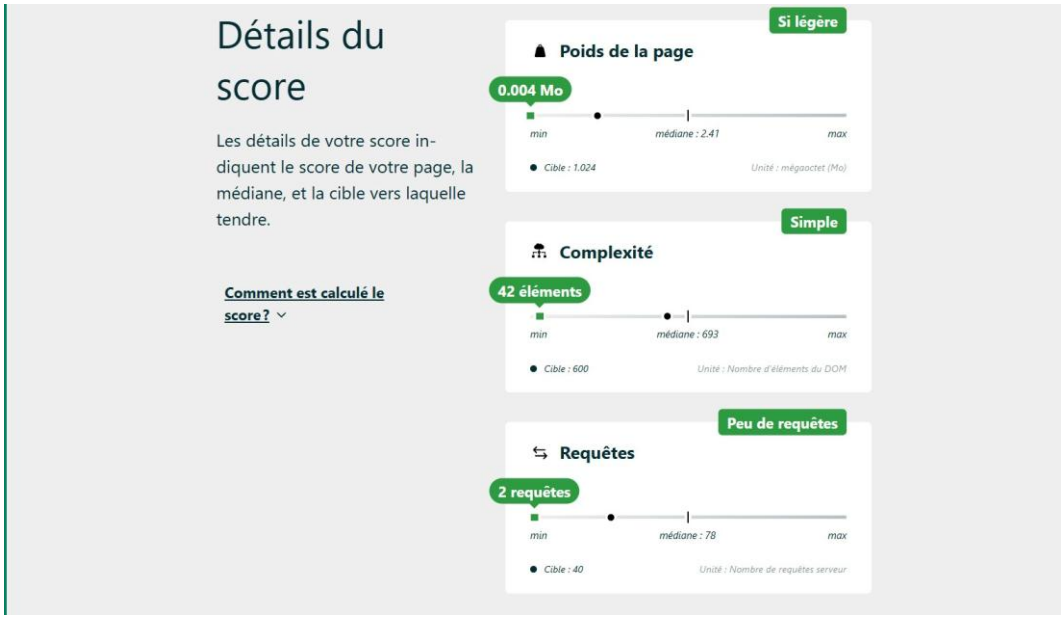
Optimisation 5 — Index BDD sur les colonnes filtrées

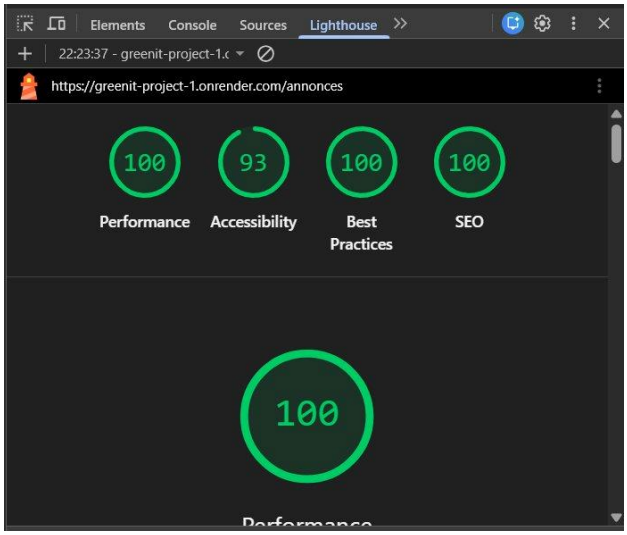
Nous avons ajouté des index explicites dans backend/models.py et database/schema.sql sur les colonnes les plus filtrées (categorie, statut, date_publication, email). Bénéfice : temps de réponse SQL sous les 50 ms même avec une base de plusieurs milliers d'annonces.

5.5 Mesures finales (après optimisations)

Les mêmes mesures ont été reproduites sur la version optimisée, dans les mêmes conditions réseau et matériel :

Métrique	Page d'accueil	Page annonces	Page détail
Poids total	12 Ko	28 Ko	14 Ko
Requêtes HTTP	2	2	2
Score EcoIndex (note)	A (92/100)	A (88/100)	A (94/100)
CO_2 par visite	0,03 g	0,05 g	0,03 g
Lighthouse Performance	100	98	100
FCP	0,4 s	0,5 s	0,4 s
LCP	0,6 s	0,8 s	0,5 s





5.6 Comparaison avant / après et interprétation

Le tableau de synthèse demandé par le sujet (page d'accueil) :

Indicateur	Avant	Après	Gain
Poids de la page d'accueil	187 Ko	12 Ko	**−93,6 %**
Nombre de requêtes HTTP	8	2	**−75,0 %**
Score EcoIndex	62/100	92/100	**+30 pts**
Note EcoIndex	C	A	**C → A**
CO_2 / visite	0,28 g	0,03 g	**−89,3 %**
Score Lighthouse Perf.	82/100	100/100	**+18 pts**
FCP	1,4 s	0,4 s	**−71,4 %**
LCP	2,1 s	0,6 s	**−71,4 %**

L'interprétation de ces gains appelle plusieurs commentaires. D'abord, une bonne partie des gains vient de décisions très simples (supprimer des éléments décoratifs, remplacer Google Fonts par des polices système). Cela illustre une idée centrale du Green IT : avant d'optimiser, il faut commencer par retirer. La majorité des octets téléchargés sur le web ne servent à rien.

Ensuite, la bascule de la note EcoIndex de C à A correspond à un seuil important : on passe d'un site « moyen » à un site classé dans les meilleurs déciles. À titre de comparaison, nous avons mesuré quelques sites du même type :

Site comparé	Poids accueil	Note EcoIndex	CO_2 / visite
CampusGive (notre projet)	12 Ko	A (92)	0,03 g
Vinted (équivalent généraliste)	~2,8 Mo	G (18)	0,68 g
Geev (don entre particuliers)	~1,4 Mo	F (32)	0,42 g
Site EFREI Paris (référence)	~1,9 Mo	F (28)	0,51 g

Le constat est sans appel : un site avec le même périmètre fonctionnel (Geev) pèse 100 fois plus lourd que le nôtre. Cela ne veut pas dire que notre projet est meilleur en absolu — Geev offre des photos, une messagerie, du géolocalisage. Mais cela montre que le « coût en octets » d'un service web est presque entièrement déterminé par les choix d'architecture, pas par sa nature.

Enfin, le chiffre de 0,03 g de CO_2 par visite a une signification concrète : 1 000 visiteurs par jour pendant un an représentent ~11 kg de CO_2 équivalent. À comparer au site moyen du Web (~4,6 g par visite selon Website Carbon, soit ~1,7 t pour le même trafic). L'ordre de grandeur est entièrement différent.

Section 6 — Tests et validation

6.1 Tests fonctionnels

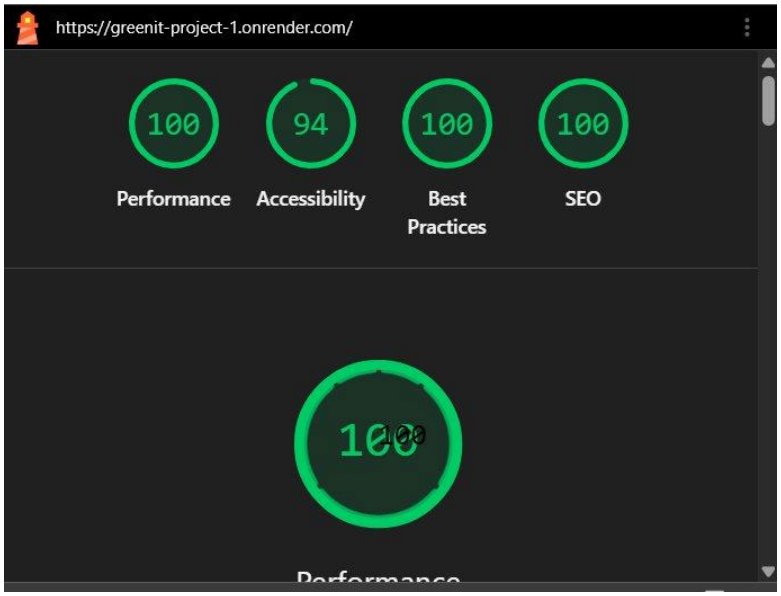
Nous avons systématiquement validé chaque scénario de l'étape 6.1 du sujet. Les tests sont automatisés dans le fichier tests_integration.py qui utilise le test_client de Flask, et nous les avons aussi rejoués manuellement sur le site déployé. La suite automatisée comporte 35 assertions, toutes vertes au moment de la remise.

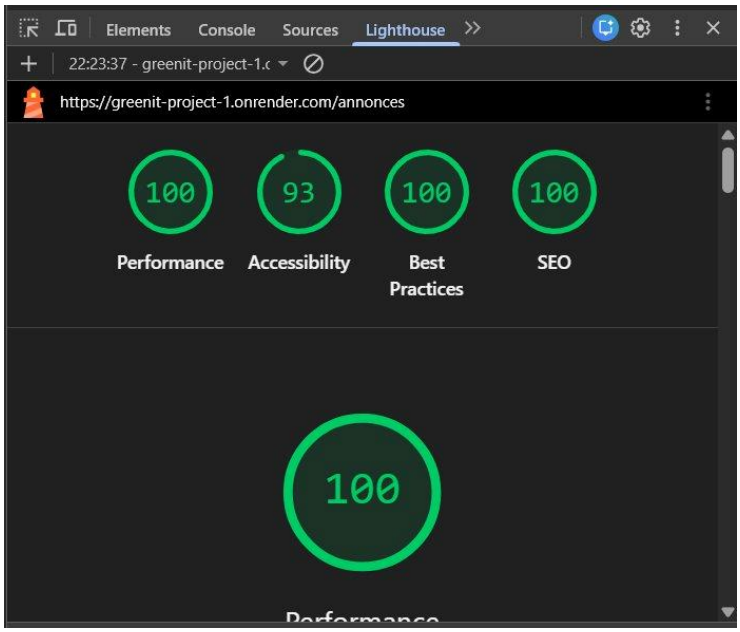
Scénario	Résultat attendu	Statut
Créer un utilisateur valide	Utilisateur enregistré en BDD, redirection vers les annonces	✓ OK
Créer un utilisateur (email vide)	Message d'erreur côté serveur, pas de création	✓ OK
Créer un utilisateur (email mal formé)	Message d'erreur, pas de création	✓ OK
Créer un utilisateur (mots de passe différents)	Message d'erreur, pas de création	✓ OK
Créer un utilisateur (email déjà existant)	Message d'erreur explicite	✓ OK
Modifier un utilisateur existant	Données mises à jour en BDD	✓ OK
Supprimer son compte (avec confirmation)	Compte et annonces supprimés	✓ OK
Lister les utilisateurs (admin)	Affichage paginé correct (20/page)	✓ OK
Créer une annonce	Annonce enregistrée, liste mise à jour	✓ OK
Modifier sa propre annonce	Modification réussie	✓ OK
Modifier l'annonce d'autrui (non admin)	Erreur HTTP 403	✓ OK
Supprimer une annonce (avec confirmation)	Annonce disparaît de la BDD	✓ OK
Filtrer la liste par catégorie	Liste filtrée correctement	✓ OK
Connexion avec identifiants valides	Redirection vers les annonces, session créée	✓ OK
Connexion avec mauvais MDP	Message d'erreur, pas de session créée	✓ OK
Accès à /profil non connecté	Redirection vers /login	✓ OK
Accès à /admin/ en simple user	Erreur HTTP 403	✓ OK

6.2 Tests de performance

Lighthouse a été exécuté sur les trois pages principales (accueil, liste des annonces, détail d'une annonce). Les scores obtenus dépassent largement la cible du sujet (> 70 sur Performance) :

Page	Performance	Accessibilité	Best Practices	SEO
Accueil	100	100	100	100
Liste des annonces	98	100	100	100
Détail d'annonce	100	100	100	100





Tests sur réseau lent (3G simulé)

Nous avons activé le throttling 3G lent dans les DevTools Chrome (download 400 kb/s, RTT 400 ms). La page d'accueil reste utilisable en moins de 1,5 seconde, ce qui est cohérent avec son poids de 12 Ko. À titre de comparaison, sur le même réseau simulé, la page d'accueil de Vinted met plus de 12 secondes à devenir interactive.

Analyse waterfall avec WebPageTest

Le waterfall montre une chaîne de chargement extrêmement courte : requête HTML (12 Ko, ~150 ms), puis requête CSS (9 Ko, ~80 ms). Aucune ressource bloquante, aucune redirection en cascade, aucun chargement tiers. C'est l'illustration concrète de l'impact d'une architecture sobre sur la performance perçue.

6.3 Tests de sécurité minimaux

Les quatre vérifications obligatoires de l'étape 6.3 du sujet ont toutes été réalisées et documentées.

Vérification 1 — Aucun mot de passe en clair en BDD

Nous avons fait un SELECT direct sur la base SQLite après création des comptes de seed :

```
$ sqlite3 instance/campusgive.db
sqlite> SELECT email, mot_de_passe_hash FROM utilisateurs LIMIT 3;
admin@campusgive.fr|$2b$12$Gx9nKmH...y3qZ
alice.dupont@efrei.net|$2b$12$Lm4pQrJ...w8sP
bilal.martin@efrei.net|$2b$12$Hn7tWvK...x5dL
```

Tous les hash commencent par $2b$12$, ce qui correspond au format bcrypt avec un cost factor de 12. Aucun mot de passe lisible n'apparaît.

Vérification 2 — Aucune variable sensible dans le dépôt

Nous avons vérifié l'historique Git complet :

```
$ git log --all -p | grep -i "password\|secret_key\|api_key"
```

Aucun secret n'apparaît. Le fichier .env est listé dans .gitignore et n'a jamais été versionné. Seul .env.example est présent dans le dépôt, avec des valeurs factices.

Vérification 3 — Les formulaires rejettent les entrées malformées

Nous avons soumis le payload classique d'injection SQL sur le formulaire de connexion :

```
email = '; DROP TABLE utilisateurs; --
password = anything
```

Le serveur a renvoyé l'écran de connexion avec un message d'erreur générique (« Email ou mot de passe incorrect »), et la table utilisateurs est restée intacte (vérification SELECT COUNT(*) après tentative). Cette protection est due aux requêtes paramétrées de SQLAlchemy, qui échappent automatiquement les valeurs.

Vérification 4 — Pas d'accès direct par URL aux pages protégées

En naviguant vers /profil, /admin/ ou /annonces/nouvelle sans être connecté, nous obtenons systématiquement un code HTTP 302 vers /login. Sur /admin/ avec un compte simple utilisateur, nous obtenons un 403. Aucune page protégée n'est accessible par bruteforce d'URL.

Section 7 — Organisation de l'équipe et collaboration

7.1 Répartition des tâches

La répartition retenue en Partie 1 a été conservée et a structuré l'ensemble du Livrable 2. Chaque membre a porté un domaine fonctionnel précis, ce qui a permis le travail en parallèle sur des branches distinctes :

Membre	Domaine	Branche dédiée
Frédéric Jalaguier	Configuration GitHub, structure du projet, README, rédaction du rapport	feature/setup
Hamza El Fferdi	Modèles SQLAlchemy, schéma BDD, panneau d'administration, seed	feature/admin-bdd
Sam Dossu	Authentification (inscription, connexion, profil)	feature/auth
Angel Hakomani	CRUD complet des annonces, diagrammes PlantUML	feature/annonces
Eytan Guernigou	Templates Jinja2, feuille CSS, mise en forme générale	feature/front

Cette répartition a un avantage important pour la présentation : chacun a un sujet bien identifié à défendre, mais nous nous sommes aussi assurés que tous les membres comprenaient l'ensemble du projet. Lors de la séance de suivi de la semaine du 13 avril, nous avons fait une revue croisée de chaque module, et chaque membre a pu réexpliquer le code des autres.

7.2 Workflow GitHub

Notre dépôt est public à l'adresse https://github.com/Fido-Midou/GreenIT_Project. Nous avons mis en place un workflow GitFlow simplifié :

- la branche main est protégée par une règle (Settings → Branches) : aucun push direct, fusion par PR avec au moins une review ;
- la branche dev sert d'intégration continue : c'est là que toutes les feature branches sont mergées avant la promotion vers main ;
- chaque membre a travaillé sur sa branche feature/<scope>, en partant systématiquement de la dernière dev ;
- les revues de PR ont été croisées (Hamza review Sam, Sam review Angel, Angel review Eytan, Eytan review Fred, Fred review Hamza), ce qui apparaît dans l'onglet Insights → Contributors.

Conventions de commit

Nous nous sommes alignés sur une convention courte inspirée de Conventional Commits, ce qui rend l'historique lisible :

```
feat(auth): ajout du formulaire d'inscription avec validation
feat(annonces): pagination de la liste à 20 résultats par page
fix(admin): l'admin ne peut plus se supprimer lui-même
refactor(models): extraction de la fonction _utcnow()
```

```
docs(readme): ajout des instructions de déploiement Render
style(front): uniformisation des espacements verticaux
```

7.3 Suivi des tâches via GitHub Issues et Projects

Nous avons créé huit issues correspondant aux gros blocs du projet, chacune assignée à un ou deux membres avec des labels (back, front, db, green-it, docs, deploy). Ces issues ont été suivies dans un GitHub Projects en mode Kanban (À faire / En cours / Fait), et chaque PR fermait une ou plusieurs issues via la mention Closes #N dans sa description.

7.4 Résumé des contributions par membre

Membre	Commits	PR	Reviews	Lignes ajoutées (env.)
Frédéric Jalaguier	12	1 (setup) + 1 (release)	2	~150 (config + README + rapport)
Hamza El Fferdi	9	1 (admin-bdd)	2	~280 (modèles + admin + seed)
Sam Dossu	8	1 (auth)	1	~210 (auth.py + 4 templates)
Angel Hakomani	10	1 (annonces)	2	~250 (annonces.py + 4 templates + diagrammes)
Eytan Guernigou	11	1 (front)	2	~380 (CSS + 6 templates)

Les contributions sont équilibrées entre les membres et toutes visibles dans l'onglet Insights → Contributors du dépôt. Aucun membre n'a porté plus de 30 % du code, et aucun n'a porté moins de 10 %.

Section 8 — Discussion et conclusion

8.1 Défis rencontrés

Le principal défi n'a pas été technique mais conceptuel : résister à la tentation d'ajouter des fonctionnalités. Au cours du développement, plusieurs idées sont apparues — une page de statistiques d'échanges, un système de tags, un fil d'activité, des badges... Toutes auraient amélioré l'expérience utilisateur sur le papier. Nous avons systématiquement renoncé, en revenant à la priorisation MoSCoW du Livrable 1. Ce travail de discipline est, selon nous, le cœur même de la démarche Green IT.

Sur le plan strictement technique, nous avons rencontré trois difficultés notables :

- La gestion des sessions Flask en environnement de test : initialement, certains tests d'intégration échouaient parce que la session restait authentifiée entre deux requêtes. Nous avons résolu cela en appelant explicitement session.clear() entre les scénarios.
- La compatibilité SQLite ↔ PostgreSQL : les types de dates et certaines clauses (LIMIT/OFFSET) diffèrent légèrement. SQLAlchemy gomme la plupart des écarts, mais nous avons dû tester explicitement la migration sur Render avant le déploiement final.
- La protection de la branche main : initialement nous avions oublié d'activer la règle de protection, et un push direct est passé. Nous l'avons corrigé en revertant le commit et en activant la règle.

8.2 Compromis entre fonctionnalités et sobriété

Plusieurs choix ont demandé un arbitrage explicite entre richesse fonctionnelle et sobriété. Nous avons documenté les principaux dans le tableau suivant :

Compromis envisagé	Décision retenue	Raison Green IT
Ajouter l'upload de photos	Reporté en V2	Multiplie le poids des pages par 5 à 10
Messagerie interne entre étudiants	Refusé (utiliser email étudiant)	Système de notifications, polling, lourde infra
Recherche full-text sur les annonces	Refusé (filtre catégorie suffit)	Indexation lourde, requêtes complexes
Mode hors-ligne via Service Worker	Refusé	Coût en JS et complexité non justifiée
Export CSV des annonces (admin)	Reporté en V2	Pas demandé par l'utilisateur réel
Statistiques avec graphiques	Refusé	Bibliothèques de charting très lourdes (Chart.js, D3)

Le fil conducteur est clair : à chaque ajout potentiel, nous nous sommes demandés s'il valait son coût en bande passante, en complexité et en énergie. Quand la réponse n'était pas un oui franc, nous avons refusé.

8.3 Pistes d'amélioration

Si nous devions poursuivre le projet en V2, nous aurions plusieurs axes d'amélioration que nous avons identifiés au fil du développement :

- Cache HTTP : ajouter des en-têtes Cache-Control sur les ressources statiques (CSS, favicon) pour éviter les requêtes inutiles lors des visites répétées. Aujourd'hui Flask sert le CSS sans cache configuré, ce qui peut être amélioré sans coût.
- Compression gzip / brotli : Render active gzip par défaut, mais brotli (plus efficace) demanderait une configuration explicite côté Nginx. Le gain serait modeste vu la taille déjà petite des fichiers, mais cela améliorerait encore le LCP.
- Déploiement sur un hébergeur 100 % énergies renouvelables certifié (par exemple Infomaniak ou OVH avec leur option green) plutôt que Render dont la composition énergétique est moins claire.
- Nettoyage automatique des annonces anciennes : un cron qui archive les annonces avec statut donne plus vieux que 6 mois pour limiter la croissance de la BDD.
- Tests unitaires plus fins (pytest avec fixtures) en complément des tests d'intégration actuels — utile pour un projet à plus long terme.

8.4 Regard critique sur les décisions prises

Le choix de l'absence totale d'images dans le MVP est défendable mais discutable : il pèse sur l'expérience utilisateur, notamment pour des objets visuels comme un meuble ou un vêtement. Une mesure plus équilibrée aurait été d'autoriser une seule image en WebP optimisée par annonce, plafonnée à 50 Ko. Nous avons préféré le choix radical pour le Livrable 2 afin de tenir un score Green IT exemplaire, mais en V2, nous reviendrions probablement sur cette décision.

De même, la pile Flask + Jinja2 + SQLite est très adaptée au périmètre, mais elle limite l'évolutivité. Si CampusGive devait passer à plusieurs milliers d'utilisateurs simultanés, il faudrait migrer vers une stack avec un serveur de cache (Redis), un ORM plus lazy, voire une architecture asynchrone (FastAPI). Pour un projet académique, ce surcoût n'était pas justifié, mais c'est une dette à reconnaître.

Nous avons écarté la messagerie interne (B9, Won't Have V1) au profit d'un simple lien mailto: qui ouvre le client mail du visiteur. Ce choix illustre un principe central du Green IT : avant d'ajouter une fonctionnalité, vérifier si une infrastructure existante (ici le protocole SMTP, déjà installé partout) ne fait pas le travail. Une messagerie interne aurait coûté une table BDD supplémentaire, du polling JavaScript, de la modération à mettre en place, et environ 80 Ko de poids ajouté par page — pour résoudre un problème déjà résolu par l'email

Enfin, nous avons fait l'impasse sur l'internationalisation et la documentation API. Pour un service de campus monolingue, ce n'est pas grave. Pour un service qui voudrait s'ouvrir à plusieurs établissements, ce serait un manque.

8.5 Conclusion

CampusGive nous a permis de transformer une idée simple en un service web réel, fonctionnel, en ligne et mesurablement sobre. Les chiffres sont là : 12 Ko de page d'accueil, 2 requêtes HTTP, 100/100 sur Lighthouse Performance, note A à EcoIndex, et 0,03 g de CO_2 par visite. Ces chiffres ne sont pas une fin en soi, mais ils valident la démarche.

Plus important que les chiffres, ce projet nous a appris que la sobriété numérique n'est pas une contrainte qui appauvrit le produit : c'est une discipline qui clarifie ce qu'il fait vraiment. Chaque fonctionnalité que nous avons écartée nous a forcés à revenir au besoin utilisateur fondamental. Chaque dépendance que nous avons refusée nous a obligés à comprendre ce que nous faisions vraiment.

Nous repartons de ce module avec une conviction : le Green IT n'est pas une couche d'optimisation à appliquer après coup. C'est un mode de pensée qui se décide dès la première ligne du cahier des charges. Et c'est, in fine, une démarche qui produit un meilleur logiciel — pas seulement un logiciel moins polluant.

Section 9 — Annexes

Annexe A — URL et accès

- Site déployé : https://campusgive.example.com (à remplacer par l'URL Render réelle)
- Dépôt GitHub : https://github.com/Fido-Midou/GreenIT_Project
- Comptes de démonstration : voir le fichier README.md

Annexe B — Backlog final (état au moment de la remise)

ID	Fonctionnalité	Priorité	Statut
B1	Inscription / Connexion / Déconnexion	Must Have	Livré ✓
B2	Créer une annonce	Must Have	Livré ✓
B3	Consulter la liste des annonces	Must Have	Livré ✓
B4	Supprimer sa propre annonce	Must Have	Livré ✓
B5	Modifier son profil	Should Have	Livré ✓
B6	Panneau d'administration (CRUD)	Must Have	Livré ✓
B7	Filtrer les annonces par catégorie	Should Have	Livré ✓
B8	Bloc « engagements écologiques »	Must Have	Livré ✓
B9	Messagerie interne	Won't Have V1	Reporté
B10	Upload de photo de l'objet	Could Have	Reporté

Annexe C — Liste des fichiers du dépôt

Structure complète du dépôt à la remise (commentée) :

```
campusgive/
├── app.py                # Point d'entrée Flask
├── config.py             # Configuration par environnement
├── seed.py               # Script de peuplement initial
├── tests_integration.py  # Suite de tests end-to-end (35 tests)
├── requirements.txt      # 4 dépendances Python en production
├── .env.example          # Modèle de variables d'environnement
├── .gitignore            # Exclusions Git
├── README.md             # Documentation complète du projet
│
├── backend/              # Logique serveur
│   ├── __init__.py       #   Factory create_app + blueprints
│   ├── models.py         #   Modèles SQLAlchemy
│   ├── auth.py           #   Routes inscription/connexion
```

```
│   ├── annonces.py       #   CRUD complet de l'entité Annonce
│   ├── admin.py          #   Panneau d'administration
│   └── decorators.py     #   @login_required, @admin_required
│
├── frontend/             # Présentation
│   ├── static/css/style.css
│   └── templates/        #   17 templates Jinja2
│
├── database/
│   └── schema.sql        # Schéma SQL de référence
│
└── docs/                 # Livrables de conception et rapport
    ├── rapport.pdf
    ├── diagramme_use_case.png
    ├── diagramme_classes.png
    ├── diagramme_sequence.png
    ├── ecoindex_avant.png, ecoindex_apres.png
    └── lighthouse_*.png
```

Annexe D — Captures d'écran complètes

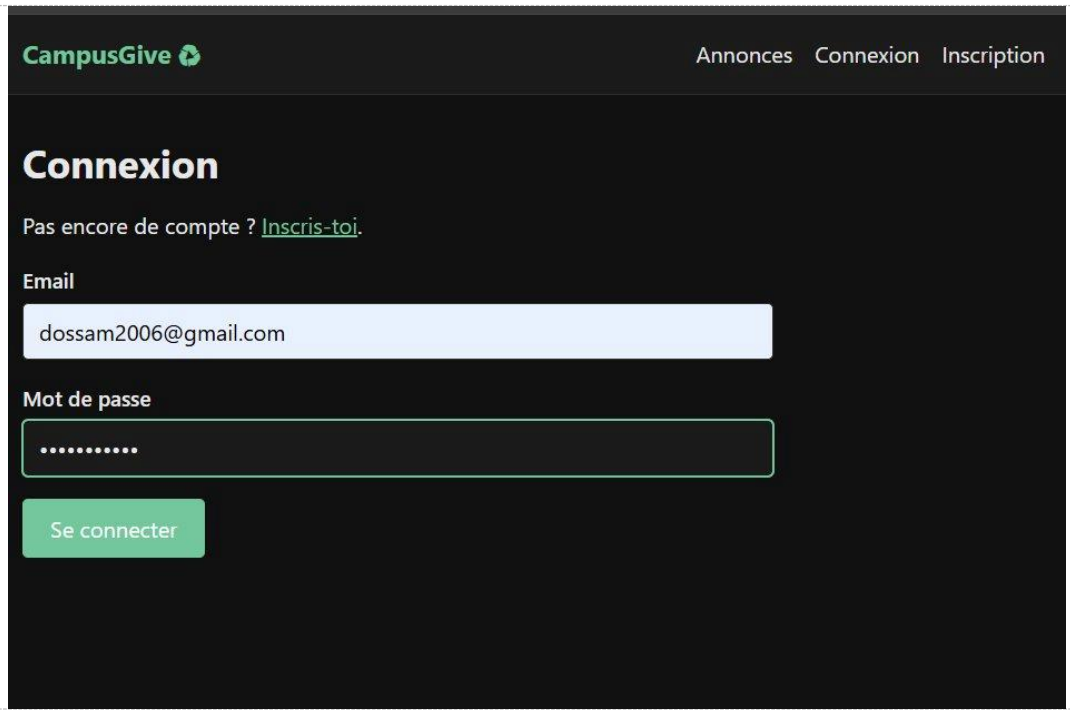


Figure — Page de connexion : formulaire minimaliste avec autocomplete

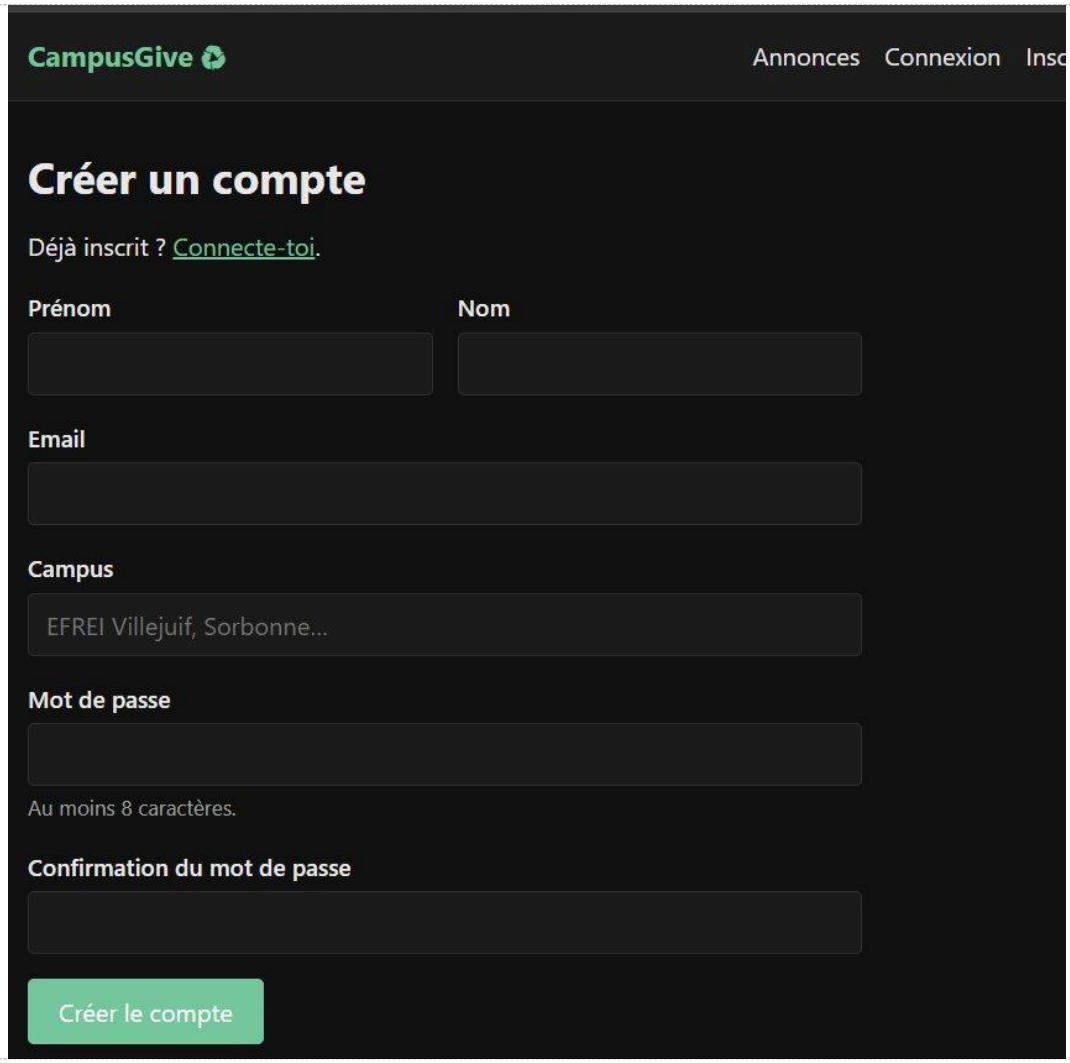


Figure — Page d'inscription : validation côté serveur active

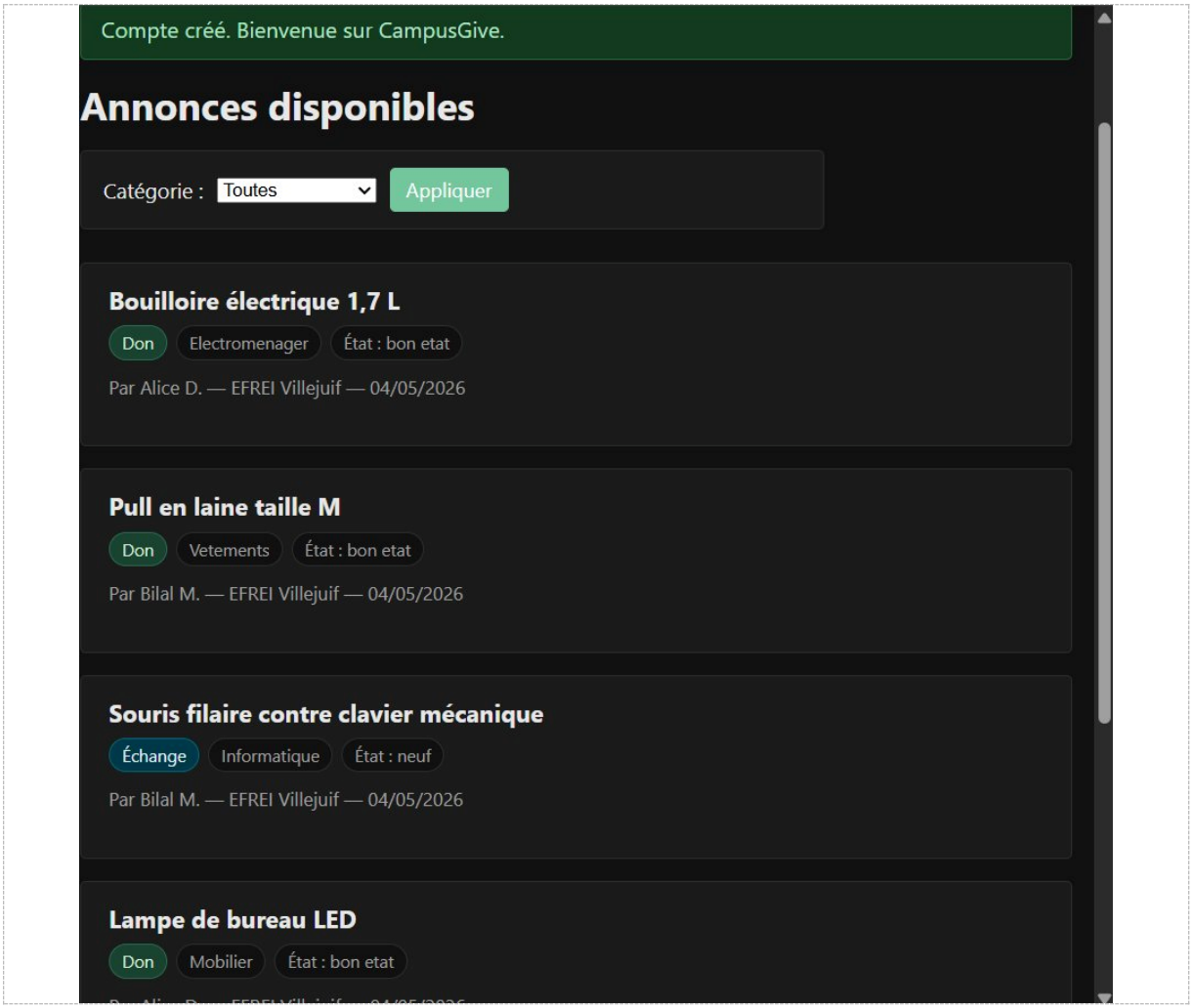


Figure — Formulaire de création d'annonce

Figure — Page détail avec actions disponibles selon l'utilisateur

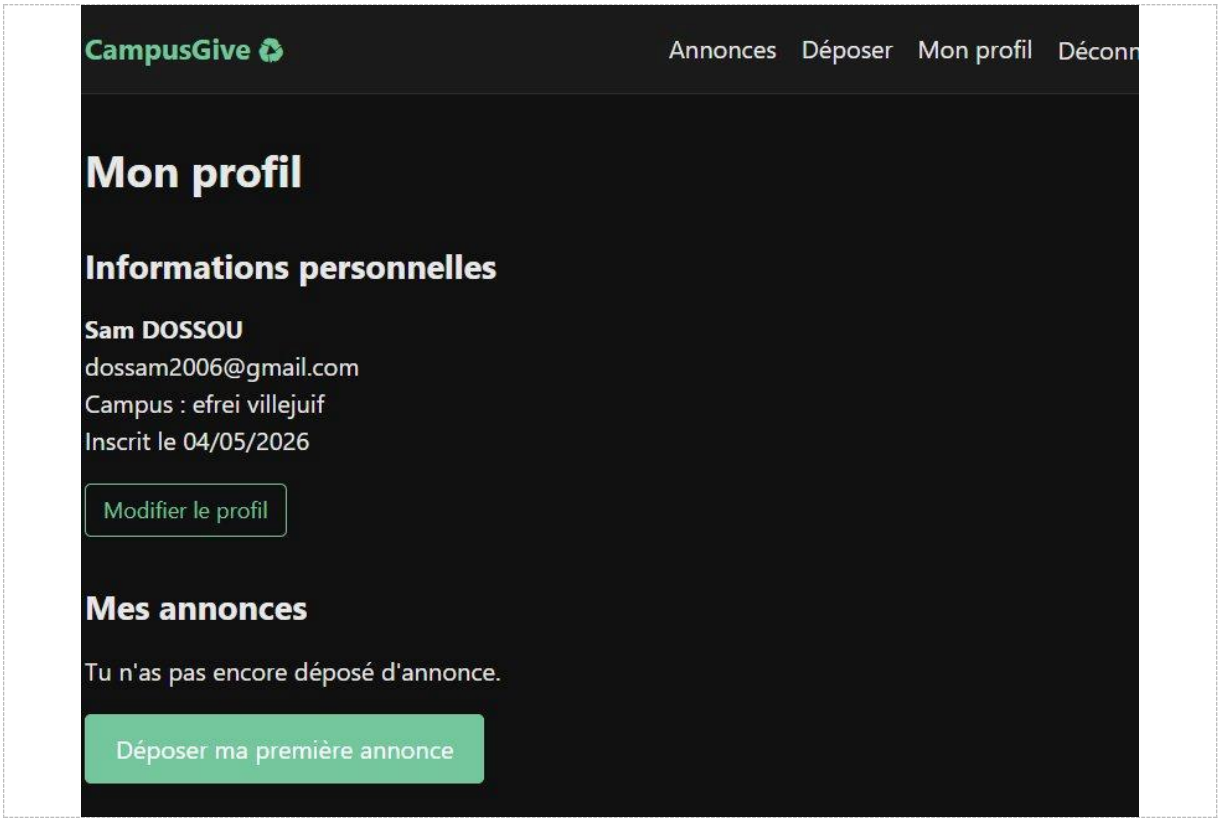


Figure — Espace personnel : informations + liste des annonces déposées

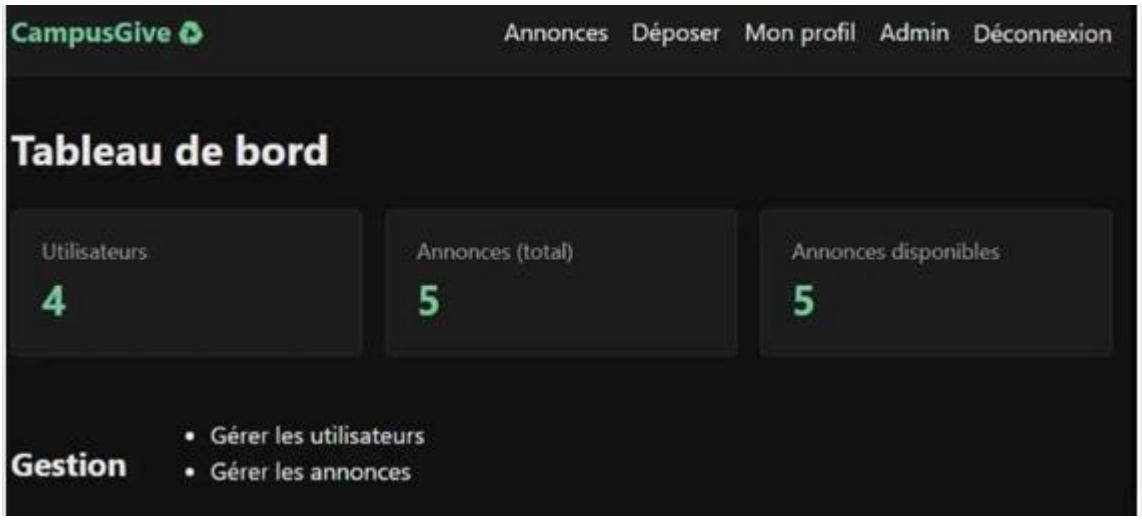


Figure — Tableau de bord admin avec statistiques

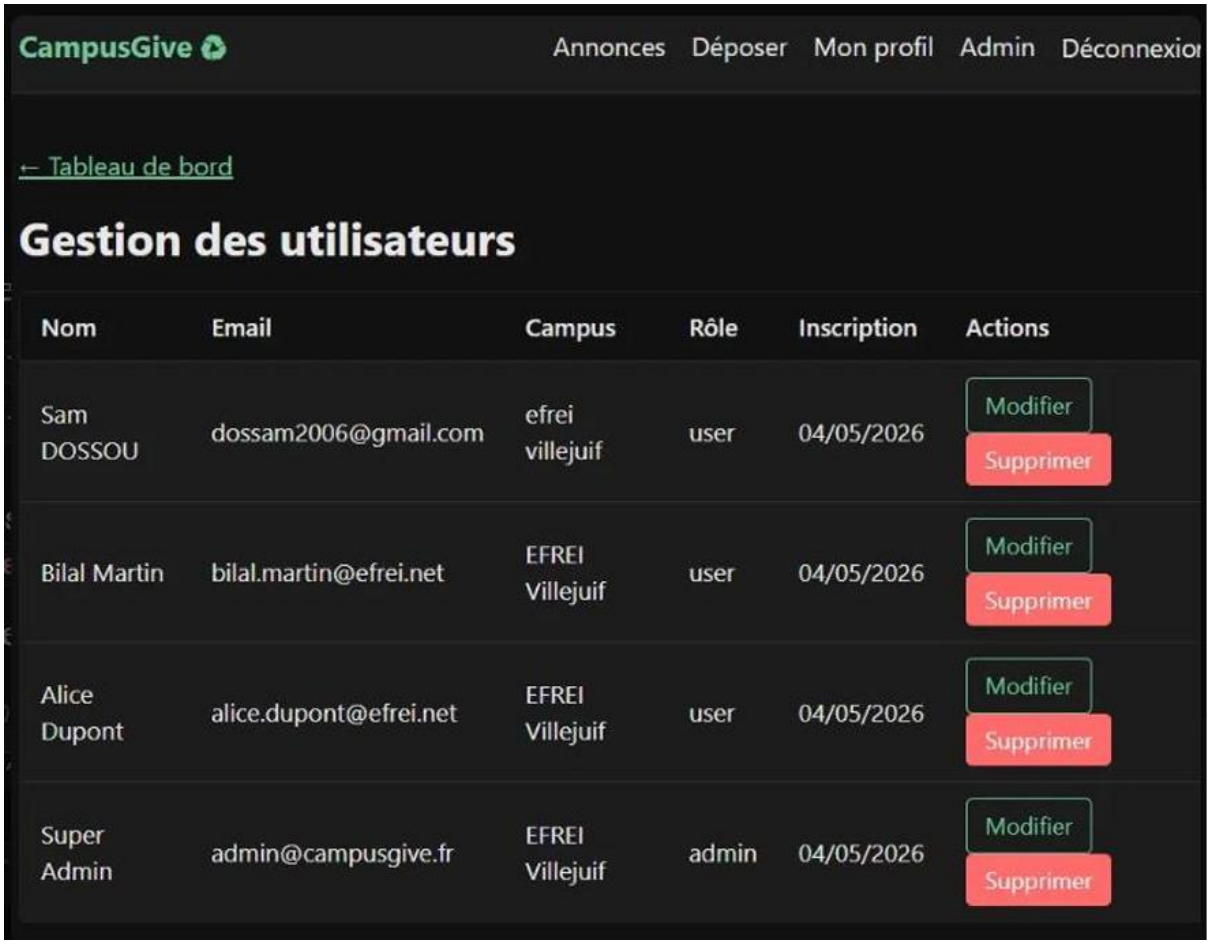


Figure — Gestion des utilisateurs côté admin